Exercise 2 — Concepts

Advanced System Theory · Week 02: System Properties and Linearization

Course: Advanced System Theory (Paderborn University, Automatic Control Group RAT, Prof. Dr. Erdal Kayacan), winter semester 2026/27 **Lecture:** Week 02 — System properties and linearization of nonlinear systems **Covers:** everything needed for Exercise 2 — Task 1 (system properties), Task 2 (equilibria and linearization), Task 3 (evolution of a discrete-time system, internal and BIBO stability)

This sheet builds every idea from the ground up and assumes only school calculus and a little linear algebra. Each test is given as a recipe you can apply mechanically, followed by worked examples. The full solutions are in `solutions.md`; the scripts in `python/` check every number.

Notation. u input, y output, x state, $t \in \mathbb{R}$ continuous time, $k \in \mathbb{Z}$ discrete time. Bold or vector quantities (x, u, A, ...) are vectors/matrices. A subscript 0 or a star (x_0, x^*) marks an operating/equilibrium point. $\Delta x = x - x_0$ is a deviation from it.

Contents

1. Signals, systems and how to manipulate signals
2. Linearity
3. Time invariance
4. Causality
5. Invertibility and stability
6. Worked lecture exercise: the property table
7. Nonlinear state-space models
8. Equilibrium points and how to find them
9. Linearization with Jacobians
10. What the linear model says about stability (2×2 toolkit)
11. Discrete-time LTI systems: solution of the state equation
12. Computing A^k with eigenvalues and eigenvectors
13. Modes, internal stability and BIBO stability
14. Cheat sheet

---

1. Signals, systems and how to manipulate signals

1.1 What a “system” is here

A **system** S is a rule that maps an input signal $u(\cdot)$ to an output signal $y(\cdot)$:

$$y = S[u].$$

The notation $u(\cdot)$ means “the whole signal”, i.e. its values at all times; $u(t)$ is one value at the time t. This distinction matters: a property like time invariance compares *whole signals*.

In Task 1 the systems are given as explicit formulas $y(t) = \ldots$ (I/O relationships). Some of them are **memoryless (static)**: $y(t)$ depends only on u at the same instant t (e.g. $y(t) = u(t)\cos(2\pi f_0 t + \phi)$). Others have **memory**: $y(t)$ depends on u at other times (e.g. $u(t+1)$ or $u(2t)$).

In Tasks 2 and 3 the systems are given as **state-space models** — differential or difference equations with a state x (§7, §11).

1.2 Time shift, time scaling, and the "replace t" rule

Three operations appear constantly. Let $u(t)$ be a signal and $T > 0$.

Operation	New signal	Picture
Delay by T	$v(t) = u(t-T)$	graph moves **right** by T (events happen later)
Advance by T	$v(t) = u(t+T)$	graph moves **left** by T (events happen earlier)
Compression by 2	$v(t) = u(2t)$	graph squeezed towards $t = 0$; runs twice as fast

Why does $u(t-T)$ move right? The value that u had at time 0 is now found where $t - T = 0$, i.e. at $t = T$.

The "replace t" rule (the single most important trick for Task 1). If $v(t) = u(2t)$ and you want v at another time, say $v(t-T)$, you replace *every* t in the definition by $t - T$:

$$v(t-T) = u\big(2(t-T)\big) = u(2t-2T).$$

The common mistake is to write $u(2t-T)$. That is a different signal: it is u first delayed by T and *then* compressed. Keep the two orders strictly apart:

- **Shift the input first, then apply the system:** feed $u_T(t) := u(t-T)$ into the system. In the formula $y(t) = u(2t)$, the system reads its input at time $2t$, so the output is $u_T(2t) = u(2t-T)$.
- **Apply the system first, then shift the output:** $y(t-T) = u(2(t-T)) = u(2t-2T)$.

The safest way to avoid errors is to give the shifted input its own name (u_T, u_2, ...) and only then substitute.

1.3 How properties are proved and disproved

All properties in this sheet are "for all" statements: *for all* inputs, *for all* shifts, *for all* scalars.

- To **prove** a property you must show it in general (with symbols, not numbers).
- To **disprove** it, a **single counterexample** is enough: one particular input, one particular T, one particular α for which it fails.

This is why the solutions show general derivations for "Yes" and one concrete counterexample for "No".

---

2. Linearity

2.1 Definition for I/O systems

A system is **linear** if it satisfies the **superposition principle**: for all inputs u_1, u_2 and all real numbers α_1, α_2,

$$S[\alpha_1 u_1 + \alpha_2 u_2] = \alpha_1 S[u_1] + \alpha_2 S[u_2].$$

This combines two properties:

- **Additivity:** $S[u_1+u_2] = S[u_1]+S[u_2]$ — the response to a sum is the sum of the responses.
- **Homogeneity (scaling):** $S[\alpha u] = \alpha S[u]$ — doubling the input doubles the output.

2.2 Definition for systems with a state (lecture Definition 1.1)

When the system has an initial state x_0, the output depends on both: $y(x_0, u, t)$. The lecture requires superposition in each argument separately, plus a decomposition:

$$\begin{aligned}
y(0,\ \alpha_1 u_1 + \alpha_2 u_2,\ t) &= \alpha_1 y(0, u_1, t) + \alpha_2 y(0, u_2, t) && \text{(linear in the input, zero initial state)} \\
y(\beta_1 x_{0,1} + \beta_2 x_{0,2},\ 0,\ t) &= \beta_1 y(x_{0,1}, 0, t) + \beta_2 y(x_{0,2}, 0, t) && \text{(linear in the initial state, zero input)} \\
y(x_0, u, t) &= y(0, u, t) + y(x_0, 0, t) && \text{(total = forced response + free response)}
\end{aligned}$$

The third line says the output splits into a **zero-state (forced) response** and a **zero-input (free) response**. You will see exactly this split in Task 3(c): $x[k] = \underbrace{A^k x_0}_{\text{free}} + \underbrace{\sum A^{k-1-i} B u[i]}_{\text{forced}}$.

Every LTI state-space model $\dot{x} = Ax + Bu,\ y = Cx + Du$ is linear in this sense.

2.3 Recipe: testing an I/O formula for linearity

1. **Quick necessary test — zero in, zero out.** Put $u \equiv 0$. A linear system must give $y \equiv 0$, because homogeneity with $\alpha = 0$ gives $S[0] = S[0 \cdot u] = 0 \cdot S[u] = 0$. If $y \not\equiv 0$, the system is **not linear** — done.
2. **General test.** Define $u_3 = \alpha_1 u_1 + \alpha_2 u_2$. Compute the output y_3 by inserting u_3 into the formula wherever u appears (with the *same* time argument). Compare with $\alpha_1 y_1 + \alpha_2 y_2$.
3. **If they differ for some choice**, give one concrete counterexample.

2.4 Typical patterns

Formula	Linear?	Reason
$y(t) = c(t)\, u(t-\tau)$, any known function $c(t)$	Yes	u appears to the first power, multiplied only by things that do not depend on u
$y(t) = u(\varphi(t))$, any time map φ (e.g. $u(2t)$, $u(-t)$)	Yes	changing the *time argument* is linear: $(\alpha_1 u_1 + \alpha_2 u_2)(\varphi(t)) = \alpha_1 u_1(\varphi(t)) + \alpha_2 u_2(\varphi(t))$
sums, integrals, derivatives of u	Yes	these operations are linear

Formula	Linear?	Reason
$y = u^2$, $u(t)u(t-1)$, $\sqrt{u}$, $\lvert u\rvert$	No	not homogeneous: $(2u)^2 = 4u^2 \neq 2u^2$
$y = \cos(u)$, $\sin(\cdots + u)$, e^u	No	nonlinear function *of the input value*
$y = 3u + 1$ (affine)	No	fails zero-in-zero-out

Key distinction (needed for Task 1 c, d): a function of *time* that multiplies the input, like $\cos(2\pi f_0 t + \phi)\, u(t)$, keeps the system linear. A nonlinear function *applied to the input value*, like $\cos(2\pi f_0 t + u(t))$, destroys linearity.

---

3. Time invariance

3.1 Definition (lecture Definition 1.2)

A system is **time-invariant (TI)** if delaying the input by T just delays the output by T, for every input and every T:

$$u(t) \;\mapsto\; y(t) \qquad \Longrightarrow \qquad u(t-T) \;\mapsto\; y(t-T).$$

The lecture writes this as $y(x_0, u(t), t-T) = y(x_0, u(t-T), t)$. In words: *the system does not care when you do the experiment* — the same input applied tomorrow gives the same output, just tomorrow. Otherwise the system is **time-variant**.

3.2 Recipe: the two-path test

1. **Path 1 — shift, then system.** Define $u_T(t) := u(t-T)$. Compute the output $y_T(t)$ of the system for input u_T: in the formula, replace $u(\text{something})$ by $u_T(\text{something}) = u(\text{something} - T)$.
2. **Path 2 — system, then shift.** Take the original output formula $y(t)$ and replace **every** t by $t - T$ (the "replace t" rule of §1.2).
3. **Compare.** If $y_T(t) = y(t-T)$ for all u, t, T, the system is TI; otherwise it is time-variant.

3.3 Red flags and patterns

- t **appears explicitly outside the input** (as in $\cos(2\pi f_0 t)$, $t\,u(t)$, $e^{-t}u(t)$): almost always time-variant. In path 1 the explicit t stays t, in path 2 it becomes $t - T$. *Exception:* if the explicit time function is constant (e.g. $f_0 = 0$), it disappears and the system can be TI.
- **Time scaling of the input** ($u(2t)$, $u(-t)$): time-variant, because a shift T of the input becomes a shift of $T/2$ (or $-T$) at the output.
- **Pure shifts of the input** ($u(t-1)$, $u(t+1)$): time-invariant.
- **State space:** $\dot{x} = Ax + Bu$ with constant matrices is TI; if $A(t)$, $B(t)$, ... depend on time, it is generally time-variant.

Example (TI). $y(t) = u(t-1)$. Path 1: $y_T(t) = u_T(t-1) = u(t-1-T)$. Path 2: $y(t-T) = u(t-T-1)$. Equal ⇒ TI.

Example (time-variant). $y(t) = t\,u(t)$. Path 1: $y_T(t) = t\,u(t-T)$. Path 2: $y(t-T) = (t-T)\,u(t-T)$. These differ (e.g. $u \equiv 1$, $T = 1$: t vs $t-1$) ⇒ time-variant.

---

4. Causality

4.1 Definition (lecture Definition 1.3)

A system is **causal** if, for any two inputs and any time T,

$$u_1(t) = u_2(t) \ \ \forall t \le T \qquad \Longrightarrow \qquad y_1(t) = y_2(t) \ \ \forall t \le T.$$

In words: the output up to time T cannot depend on what the input does *after* T. **The output may depend on present and past inputs, never on future inputs.** Every physical system that runs in real time is causal: it cannot react to something that has not happened yet.

4.2 Recipe

For the output at time t, list every time instant at which the input is read. Call them the "read times" $s(t)$.

- If **all** read times satisfy $s(t) \le t$ **for all** t ⇒ **causal**.
- If **some** read time satisfies $s(t) > t$ for **some** t ⇒ **non-causal**. Counterexample: take two inputs that agree up to t but differ just after it.

Memoryless systems read only $s(t) = t$, so **every memoryless system is causal**.

4.3 Patterns

Read time	Causal?	Comment
$u(t)$, $u(t-1)$, $\int_{-\infty}^{t} u(\tau)\,d\tau$	Yes	present and past only
$u(t+1)$	No	needs the input one second in the future
$u(2t)$	No	for $t > 0$: $2t > t$, i.e. the future (for $t < 0$ it reads the past, but causality must hold for *all* t)
$u(-t)$	No	for $t < 0$: $-t > t$
$\sum_{k=-\infty}^{n} u[k]$	Yes	discrete-time accumulator

Non-causal does not mean useless: in offline signal processing (a recorded file) the "future" samples are available. It only means the system cannot run in real time.

4.4 State-space models are causal

$\dot{x}(t) = f(x(t), u(t))$ is integrated forwards: $x(t) = x(t_0) + \int_{t_0}^{t} f(x(\tau), u(\tau))\,d\tau$ uses inputs only up to t. Therefore every state-space model (in particular every LTI model) is causal by construction, as the lecture notes.

---

5. Invertibility and stability

These two are not asked for in Task 1, but they are part of Lecture 2 and of the lecture exercise in §6.

5.1 Invertibility (lecture Definition 1.4)

A system is **invertible** if distinct inputs give distinct outputs, so that the input can be recovered from the output by an inverse system.

- $y = 2u$ is invertible, inverse $w = 0.5\,y$.
- $y = u^2$ is not invertible (u and $-u$ give the same output).
- $y(t) = u(t)\cos(2\pi f_0 t + \phi)$ is not invertible: at the zeros of the cosine the input value is lost.

5.2 Stability (lecture Definition 1.5 and slide 10)

Informally, a system is **stable** if small perturbations lead to responses that do not diverge — like a ball in a valley, which returns after being pushed; a ball on a hilltop rolls away (unstable).

Two precise versions are used in the course:

- **BIBO stability (external, I/O).** Every **b**ounded **i**nput produces a **b**ounded **o**utput: if $|u(t)| \leq M_u < \infty$ for all t, then $|y(t)| \leq M_y < \infty$ for all t. (Detailed in week 10.)
- **Internal (Lyapunov-type) stability.** With zero input, the state returns to (or stays near) the equilibrium after a perturbation of the initial state. (Detailed in week 9.) For linear systems this is decided by the eigenvalues of A (§10, §13).

Lecture question: $y(t) = 0.5e^t u(t)$. The bounded input $u \equiv 1$ gives $y = 0.5e^t \to \infty$. Not BIBO stable.

BIBO recipe for I/O formulas. Assume $|u| \leq M$. Try to bound $|y|$ using $|\cos| \leq 1$, the triangle inequality $|a+b| \leq |a| + |b|$, etc. If a bound exists ⇒ BIBO stable. If you find one bounded input with an unbounded output (sums that grow, factors like e^t or t) ⇒ not BIBO stable.

---

6. Worked lecture exercise: the property table

Slides 11–13 ask for causality, stability, time invariance and linearity of four systems (x is the input here). Working them through is the best preparation for Task 1.

(1) $y[n] = x[n-4] + x[4-n]$

- *Causal?* Read times $n-4$ and $4-n$. For $n=0$: $4-n = 4 > 0$ (future). **No.**
- *Stable?* $|y| \leq |x[n-4]| + |x[4-n]| \leq 2M$. **Yes.**
- *TI?* Path 1: $x_T[n] = x[n-T]$ gives $x[n-4-T] + x[4-n-T]$. Path 2: $y[n-T] = x[n-T-4] + x[4-n+T]$. Second terms differ ($-T$ vs $+T$) because of the time reversal. **No.**
- *Linear?* Only time arguments are changed. **Yes.**

(2) $y(t) = 3x(t) + x(t-1)$ — present and past ⇒ causal; $|y| \leq 4M$ ⇒ stable; pure shifts with constant coefficients ⇒ TI; first powers only ⇒ linear. **Y Y Y Y.**

(3) $y(t) = 3x(t)x(t-1)$ — causal, stable ($|y| \leq 3M^2$), TI; but $3(2x)(2x) = 4 \cdot 3xx \neq 2 \cdot 3xx$ ⇒ **not linear**. **Y Y Y N.**

(4) $y[n] = \sum_{k=-\infty}^{n} x[k]$ **(accumulator)** — past and present ⇒ causal. The bounded input $x[k] = 1$ for $k \geq 0$ (0 before) gives $y[n] = n+1 \to \infty$ ⇒ **not stable**. Path 1: $\sum_{k \leq n} x[k-T] = \sum_{j \leq n-T} x[j]$; path 2: $y[n-T] = \sum_{k \leq n-T} x[k]$; equal ⇒ TI. Sum is linear ⇒ linear. **Y N Y Y.**

---

7. Nonlinear state-space models

7.1 The general form

A nonlinear, time-invariant system is written as

$$\dot{x} = f(x, u), \qquad y = g(x, u),$$

with state $x \in \mathbb{R}^n$, input $u \in \mathbb{R}^m$, output $y \in \mathbb{R}^p$, and $f : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$ continuously differentiable. Written out for $n = 2$:

$$\dot{x}_1 = f_1(x_1, x_2, u), \qquad \dot{x}_2 = f_2(x_1, x_2, u).$$

A system **without input** ($\dot{x} = f(x)$, as in Task 2a and 2b) is called **autonomous**.

7.2 From an n-th order ODE to state space

Many models arrive as one high-order differential equation in the output, like the pendulum $ml^2\ddot{\theta} + b\dot{\theta} + mgl \sin\theta = 0$ or Task 2c. The recipe:

1. **Solve for the highest derivative.** For a second-order equation: $\ddot{y} = h(y, \dot{y}, u)$.
2. **Choose the states** as the output and its derivatives up to one order below the highest: $x_1 = y$, $x_2 = \dot{y}$ (for order n: $x_i = y^{(i-1)}$, $i = 1..n$).
3. **Write one first-order equation per state:**
   - $\dot{x}_1 = \dot{y} = x_2$ (this is just the definition of x_2),
   - $\dot{x}_2 = \ddot{y} = h(x_1, x_2, u)$ (the ODE, with $y \to x_1$, $\dot{y} \to x_2$).
4. **Output equation:** $y = x_1$, so $g(x, u) = x_1$.

Pendulum example (slide 18). $\ddot{\theta} = -\frac{b}{ml^2}\dot{\theta} - \frac{g}{l}\sin\theta$, so with $x_1 = \theta$, $x_2 = \dot{\theta}$:

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = -\frac{b}{ml^2}x_2 - \frac{g}{l}\sin x_1.$$

---

8. Equilibrium points and how to find them

8.1 Definition (lecture Definition 1.6)

(x_0, u_0) is an **equilibrium** of $\dot{x} = f(x, u)$ if

$$f(x_0, u_0) = 0.$$

Then $\dot{x} = 0$: if the state starts at x_0 and the input is held at u_0, the state stays at x_0 forever. For an autonomous system the condition is simply $f(x_0) = 0$.

Finding equilibria means solving n (usually nonlinear) equations $f_1 = 0, \dots, f_n = 0$ for the n unknowns $x_1, \dots, x_n$. There may be none, one, several, infinitely many isolated ones, or a whole continuum.

8.2 Strategy

1. **Look for an equation with only one unknown** (or that factorises) and solve it first; substitute into the others.
2. **Use trigonometric facts:**
   - $\sin z = 0 \iff z = k\pi$, $k \in \mathbb{Z}$.
   - $\cos z = 0 \iff z = \frac{\pi}{2} + k\pi$.
   - $\cos(k\pi) = (-1)^k$ (i.e. $+1$ for even k, -1 for odd k); $\sin(k\pi) = 0$.
   - $|\sin z| \leq 1$, $|\cos z| \leq 1$, sin is odd, cos is even.
   - $1 - \cos z = 2\sin^2(z/2) \geq 0$, and $1 - \cos z \leq z^2/2$.
3. **Always check the obvious candidate $x = 0$.**
4. **Combine equations** (add, subtract) to cancel the hard terms.
5. **Bound the solutions** when the equations cannot be solved in closed form: e.g. if $a^2 + b^2 = -b\cos a$, then $a^2 + b^2 \leq |b|$, which confines (a, b) to a small disk. Inside a small region, signs of terms become known (e.g. $\cos a > 0$ for $|a| < \pi/2$), and this often rules out all but a few points.
6. **Verify** each candidate by substituting it back into **all** equations.
7. **For a system with input**, the equilibrium condition involves u_0 too. Often one asks "for which constant u_0 do equilibria exist?" (see Task 2c).

8.3 Examples from the lecture

- *Pendulum:* $f_1 = x_2 = 0$ and $f_2 = -\frac{g}{l}\sin x_1 = 0 \Rightarrow (x_1, x_2) = (k\pi, 0)$: infinitely many isolated equilibria. Even k: hanging down; odd k: upright.
- *Cart-pendulum:* $x_3 = x_4 = 0$ (velocities zero); with $u_0 = 0$, the third and fourth equations need $\sin x_2 = 0$; x_1 (cart position) is free. So $x_0 = 0$, $u_0 = 0$ is one equilibrium of a continuum.

---

9. Linearization with Jacobians

9.1 Why linearize?

Nonlinear systems have no general closed-form solution and can behave very differently depending on where they start. A linear model gives access to all linear tools: eigenvalues, modes, stability tests, controllability, observability, controller design (LQR, pole placement, observers). Linearization replaces f **locally**, near an operating point, by its tangent (plane) — never globally.

9.2 One variable: the tangent line

For $f : \mathbb{R} \to \mathbb{R}$, Taylor's theorem around x_0 gives

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \tfrac{1}{2}f''(x_0)(x - x_0)^2 + \ldots$$

Dropping the quadratic and higher terms leaves the tangent line $f(x) \approx f(x_0) + f'(x_0)(x - x_0)$. The error is of order $(x - x_0)^2$: tiny for x close to x_0, growing as we move away. Example: $\sin x \approx x$ near 0 (since $\sin 0 = 0$, $\cos 0 = 1$); $\sin x \approx -(x - \pi)$ near π (since $\cos \pi = -1$).

9.3 Several variables: partial derivatives and the Jacobian

A **partial derivative** $\partial f_i / \partial x_j$ is the ordinary derivative of f_i with respect to x_j, treating all other variables as constants.

Rules you need (with x_2 treated as a constant when differentiating w.r.t. x_1):

Expression	$\partial/\partial x_1$	$\partial/\partial x_2$
$x_1^2 + x_2^2$	$2x_1$	$2x_2$
$x_2 \cos x_1$	$-x_2 \sin x_1$	$\cos x_1$
$x_1 \sin x_2$	$\sin x_2$	$x_1 \cos x_2$
$(1 + x_1)x_1 = x_1 + x_1^2$	$1 + 2x_1$	0
$(3 + x_2^2)x_2 = 3x_2 + x_2^3$	0	$3 + 3x_2^2$
$(1 + x_1 + x_1^2)\, u$	$(1 + 2x_1)\, u$	0

(Product rule: $(ab)' = a'b + ab'$; chain rule: $(\sin(h))' = \cos(h)\, h'$.) Tip: multiply out products before differentiating — fewer mistakes.

The **Jacobian matrices** collect all first partial derivatives; row i belongs to f_i, column j to the variable:

$$\frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{n \times n}, \qquad \frac{\partial f}{\partial u} = \begin{bmatrix} \frac{\partial f_1}{\partial u_1} & \cdots & \frac{\partial f_1}{\partial u_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial u_1} & \cdots & \frac{\partial f_n}{\partial u_m} \end{bmatrix} \in \mathbb{R}^{n \times m}.$$

The multivariable Taylor expansion to first order is

$$f(x, u) \approx f(x_0, u_0) + \left.\frac{\partial f}{\partial x}\right|_{(x_0, u_0)} (x - x_0) + \left.\frac{\partial f}{\partial u}\right|_{(x_0, u_0)} (u - u_0).$$

9.4 The linearized model in deviation variables

Define the deviations $\Delta x = x - x_0$, $\Delta u = u - u_0$, $\Delta y = y - y_0$ with $y_0 = g(x_0, u_0)$. Because x_0 is constant, $\frac{d}{dt}\Delta x = \dot{x}$, and therefore (slide 21)

$$\frac{d}{dt}\Delta x = \dot{x} = f(x, u) \approx \underbrace{f(x_0, u_0)}_{= 0 \text{ at an equilibrium}} + A\,\Delta x + B\,\Delta u,$$

$$\boxed{\Delta \dot{x} = A\,\Delta x + B\,\Delta u, \qquad \Delta y = C\,\Delta x + D\,\Delta u}$$

$$A = \left.\frac{\partial f}{\partial x}\right|_{(x_0, u_0)}, \quad B = \left.\frac{\partial f}{\partial u}\right|_{(x_0, u_0)}, \quad C = \left.\frac{\partial g}{\partial x}\right|_{(x_0, u_0)}, \quad D = \left.\frac{\partial g}{\partial u}\right|_{(x_0, u_0)}.$$

Why equilibria are the natural choice: at an equilibrium the constant term $f(x_0, u_0)$ vanishes, so the deviation model is a genuine linear (not affine) system. At a non-equilibrium operating point a constant offset $f(x_0, u_0)$ remains.

9.5 Recipe

1. Write the model as $\dot{x} = f(x,u)$, $y = g(x,u)$ (§7.2).
2. Find the equilibria (§8).
3. Compute the Jacobians **symbolically** (as functions of x, u).
4. **Substitute** each equilibrium to get the numerical A, B, C, D. (Differentiate first, substitute second — never the other way round.)
5. State the model in deviation variables, and say which equilibrium it belongs to.
6. Optionally interpret: eigenvalues of A (§10).

9.6 Lecture examples

Pendulum at $(k\pi, 0)$. $\frac{\partial f}{\partial x} = \begin{bmatrix} 0 & 1 \\ -\frac{g}{l}\cos x_1 & -\frac{b}{ml^2} \end{bmatrix}$, and $\cos(k\pi) = (-1)^k$:

$$A_{\text{down}} = \begin{bmatrix} 0 & 1 \\ -\frac{g}{l} & -\frac{b}{ml^2} \end{bmatrix} \ (k \text{ even}), \qquad A_{\text{up}} = \begin{bmatrix} 0 & 1 \\ +\frac{g}{l} & -\frac{b}{ml^2} \end{bmatrix} \ (k \text{ odd}).$$

For $b = 0$, A_{down} is the slide-23 model (small-angle $\sin\theta \approx \theta$).

Cart-pendulum at $x_0 = 0$, $u_0 = 0$ *(slides 24–26).* At $x_2 = 0$: $\sin x_2 = 0$, $\cos x_2 = 1$, and every term with x_4^2 has zero derivative at $x_4 = 0$ (derivative $2x_4 = 0$). Result:

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{mg}{M} & 0 & 0 \\ 0 & -\frac{(M+m)g}{lM} & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ \frac{1}{M} \\ -\frac{1}{lM} \end{bmatrix}, \quad C = \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix}, \quad D = 0.$$

9.7 Validity

The linear model is only valid **locally**: for small $\|\Delta x\|$ and $\|\Delta u\|$. Each equilibrium has its *own* linear model, and they can behave completely differently (pendulum down: oscillation; up: falls over). The figures in `python/figures/` show nonlinear and linearized trajectories lying on top of each other near the equilibrium and separating further away.

---

10. What the linear model says about stability (2×2 toolkit)

Task 2 asks only for the linearization, but interpreting it is what linearization is for, and you need eigenvalues for Task 3 anyway.

10.1 Eigenvalues of a 2×2 matrix

The eigenvalues λ of $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ solve the **characteristic equation**

$$\det(\lambda I - A) = (\lambda - a)(\lambda - d) - bc = \lambda^2 - \operatorname{tr}(A)\,\lambda + \det(A) = 0,$$

with $\operatorname{tr} A = a + d$ and $\det A = ad - bc$. So

$$\lambda_{1,2} = \frac{\operatorname{tr} A \pm \sqrt{(\operatorname{tr} A)^2 - 4 \det A}}{2}, \qquad \lambda_1 + \lambda_2 = \operatorname{tr} A, \quad \lambda_1 \lambda_2 = \det A.$$

For a **diagonal or triangular** matrix, the eigenvalues are simply the diagonal entries.

10.2 Continuous time: sign of the real parts

For $\Delta\dot{x} = A\Delta x$ the solution is built from terms $e^{\lambda t}$ (week 3). Hence:

Eigenvalues of A	Behaviour of Δx	Name (2D)
all Re $\lambda < 0$	decays to 0	stable node / focus
some Re $\lambda > 0$	grows	unstable (saddle if the other is < 0)
some Re $\lambda = 0$, rest < 0	neither decays nor necessarily grows	marginal case

Quick 2×2 test: **both eigenvalues have negative real part ⇔** $\operatorname{tr} A < 0$ **and** $\det A > 0$**.** $\det A < 0$ ⇔ real eigenvalues of opposite sign ⇔ saddle.

10.3 Linearization and local stability of the nonlinear system (preview of week 9)

Lyapunov's indirect method: if all eigenvalues of the Jacobian A at an equilibrium have Re $\lambda < 0$, the equilibrium of the *nonlinear* system is locally asymptotically stable; if at least one has Re $\lambda > 0$, it is unstable. If some eigenvalue has Re $\lambda = 0$ (and none positive), the linearization is **inconclusive** — the neglected higher-order terms decide. Task 2c is such a case.

---

11. Discrete-time LTI systems: solution of the state equation

11.1 The model

$$x[k+1] = A\,x[k] + B\,u[k], \qquad y[k] = C\,x[k] + D\,u[k], \qquad k \in \mathbb{Z}.$$

Unlike a differential equation, a difference equation is a **recipe you can run**: given $x[0]$ and the inputs, compute $x[1]$, then $x[2]$, and so on.

Reminder of matrix–vector multiplication: $\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} p \\ q \end{bmatrix} = \begin{bmatrix} ap + bq \\ cp + dq \end{bmatrix}$ (row times column).

11.2 Free response ($u \equiv 0$)

$x[1] = Ax[0]$, $x[2] = Ax[1] = A^2x[0]$, ..., so

$$x[k] = A^k x[0], \qquad A^0 = I.$$

11.3 Full response — derivation

Iterate with the input:

$$\begin{aligned} x[1] &= Ax[0] + Bu[0] \\ x[2] &= Ax[1] + Bu[1] = A^2x[0] + ABu[0] + Bu[1] \\ x[3] &= Ax[2] + Bu[2] = A^3x[0] + A^2Bu[0] + ABu[1] + Bu[2]. \end{aligned}$$

The pattern: the input applied at step i has been "processed" by A exactly $k-1-i$ times by step k. Hence

$$\boxed{x[k] = \underbrace{A^k x[0]}_{\text{free (zero-input) response}} + \underbrace{\sum_{i=0}^{k-1} A^{k-1-i} B\, u[i]}_{\text{forced (zero-state) response}}}, \qquad k \geq 1.$$

Proof by induction. For $k = 1$ it reads $x[1] = Ax[0] + Bu[0]$ ✓. If it holds for k, then $x[k+1] = Ax[k] + Bu[k] = A^{k+1}x[0] + \sum_{i=0}^{k-1} A^{k-i}Bu[i] + A^0 Bu[k] = A^{k+1}x[0] + \sum_{i=0}^{k} A^{k-i}Bu[i]$ ✓.

This is exactly the split $y(x_0, u) = y(x_0, 0) + y(0, u)$ of the linearity definition (§2.2). The sum is a **discrete convolution** of the input with $A^j B$ (week 3).

11.4 Impulse response

With $x[0] = 0$ and the unit impulse $u[k] = \delta[k]$ (1 at $k = 0$, 0 otherwise), only the $i = 0$ term survives:

$$x[k] = A^{k-1}B \ (k \geq 1), \quad x[0] = 0, \qquad y[k] = CA^{k-1}B \ (k \geq 1), \quad y[0] = D.$$

---

12. Computing A^k with eigenvalues and eigenvectors

12.1 Eigenvalues and eigenvectors

$v \neq 0$ is an **eigenvector** of A with **eigenvalue** λ if

$$Av = \lambda v.$$

A acts on v just by scaling it. Applying A k times: $A^k v = \lambda^k v$. **This is the whole trick.**

How to compute them:

1. Eigenvalues: solve $\det(\lambda I - A) = 0$ (§10.1).
2. For each λ: solve $(A - \lambda I)v = 0$. The matrix $A - \lambda I$ is singular, so its rows are multiples of each other; one row $\begin{bmatrix} p & q \end{bmatrix}$ gives $pv_1 + qv_2 = 0$, and $v = \begin{bmatrix} q \\ -p \end{bmatrix}$ (or any multiple) works.
3. Check: $Av = \lambda v$.

12.2 Method 1 — decompose the initial state

If $A \in \mathbb{R}^{2\times 2}$ has two distinct eigenvalues $\lambda_1 \neq \lambda_2$ with eigenvectors v_1, v_2, these form a basis: every $x[0]$ can be written as $x[0] = c_1 v_1 + c_2 v_2$ (solve a 2×2 linear system for c_1, c_2). Then

$$x[k] = A^k x[0] = c_1 \lambda_1^k \, v_1 + c_2 \lambda_2^k \, v_2.$$

12.3 Method 2 — diagonalization

Put the eigenvectors as columns in $V = \begin{bmatrix} v_1 & v_2 \end{bmatrix}$ and $\Lambda = \text{diag}(\lambda_1, \lambda_2)$. Then $AV = V\Lambda$, so $A = V\Lambda V^{-1}$ and

$$A^k = (V\Lambda V^{-1})(V\Lambda V^{-1}) \cdots (V\Lambda V^{-1}) = V\Lambda^k V^{-1}, \qquad \Lambda^k = \begin{bmatrix} \lambda_1^k & 0 \\ 0 & \lambda_2^k \end{bmatrix}.$$

(All inner $V^{-1}V = I$ cancel.) **2×2 inverse:** $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \dfrac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$.

Equivalently, with $w_i^\top$ the rows of V^{-1}: $A^k = \lambda_1^k v_1 w_1^\top + \lambda_2^k v_2 w_2^\top$ (sum of rank-one "mode" matrices).

Always **check** a closed form at $k = 0$ (must give I or $x[0]$) and $k = 1$ (must give A or $Ax[0]$).

12.4 Powers of negative numbers

$(-1)^k$ alternates $1, -1, 1, -1, \ldots$ (bounded, never converges). $(-2)^k$ alternates sign and doubles in size: $1, -2, 4, -8, 16, \ldots$ In general $|\lambda^k| = |\lambda|^k$.

12.5 A useful shortcut

If B (or $x[0]$) happens to be an eigenvector, i.e. $AB = \lambda B$, then $A^j B = \lambda^j B$ for every j — no matrix powers needed. Look for this before computing (Task 3 c, d).

---

13. Modes, internal stability and BIBO stability

13.1 Modes in discrete time

From §12.2, the free response is a sum of **modes** $c_i \lambda_i^k v_i$. Each mode behaves according to its eigenvalue:

λ (real)	λ^k	mode
$0 < \lambda < 1$	$\to 0$ monotonically	decays
$-1 < \lambda < 0$	$\to 0$ with alternating sign	decays, oscillating
$\lambda = 1$	1	constant
$\lambda = -1$	± 1 alternating	bounded, does **not** converge
$\lambda > 1$	$\to \infty$	diverges
$\lambda < -1$	$\pm\infty$ alternating	diverges, oscillating

(For complex λ, $|\lambda|$ decides the growth and the angle gives a rotation.) **The boundary in discrete time is the unit circle** $|\lambda| = 1$, whereas in continuous time it is the imaginary axis $\operatorname{Re}\lambda = 0$.

13.2 Internal stability

Internal stability is about the free response $x[k] = A^k x[0]$, $u \equiv 0$:

- **Asymptotically stable:** $x[k] \to 0$ for every $x[0]$ ⇔ all $|\lambda_i| < 1$.
- **Marginally (Lyapunov) stable:** $x[k]$ bounded for every $x[0]$ — requires all $|\lambda_i| \leq 1$ (and eigenvalues on the circle not "repeated with a Jordan block" — week 9).
- **Unstable:** some $x[0]$ makes $x[k]$ unbounded ⇔ some $|\lambda_i| > 1$ (or a defective eigenvalue on the circle).

For a **specific** $x[0]$, convergence depends on which modes it excites: if $c_i = 0$ for every bad eigenvalue, those modes never appear. The answer "does it converge for $x[0] = \ldots$?" therefore needs the decomposition $x[0] = \sum c_i v_i$.

13.3 BIBO stability

BIBO stability is about the input–output behaviour from $x[0] = 0$. For an LTI system it holds if and only if the impulse response is absolutely summable, $\sum_k \|CA^{k-1}B\| < \infty$ (week 10). In particular: **if the response to the bounded input $\delta[k]$ is unbounded, the system is not BIBO stable.**

Internal vs BIBO. Internal (asymptotic) stability implies BIBO stability, but not conversely: an unstable mode that the input cannot excite (or the output cannot see) leaves the I/O behaviour bounded. In Task 3 both fail, because B lies exactly on the unstable eigenvector.

---

14. Cheat sheet

System properties (I/O formula $y(t) = \ldots$)

Property	Test	Quick signs of failure
Linear	$S[\alpha_1 u_1 + \alpha_2 u_2] = \alpha_1 S[u_1] + \alpha_2 S[u_2]$; first check $S[0] = 0$	u^2, $u \cdot u$, $\cos(u)$, e^u, constant offsets
Time-invariant	shift-then-system = system-then-shift; replace **every** t by $t - T$ in path 2	explicit t outside u (e.g. $\cos 2\pi f_0 t$), time scaling $u(2t)$, reversal $u(-t)$
Causal	all read times $s(t) \le t$ for all t	$u(t+1)$, $u(2t)$, $u(-t)$
BIBO stable	$\|u\| \le M \Rightarrow \|y\| \le M'$	accumulation, e^t, $t\,u(t)$

Memoryless ⇒ causal. Time-only factors ($c(t)u(t)$) keep linearity. Changing only time arguments keeps linearity.

Linearization

$$f(x_0, u_0) = 0, \qquad \Delta\dot{x} = A\Delta x + B\Delta u, \quad \Delta y = C\Delta x + D\Delta u, \qquad A = \tfrac{\partial f}{\partial x}\big|_0,\ B = \tfrac{\partial f}{\partial u}\big|_0,\ C = \tfrac{\partial g}{\partial x}\big|_0,\ D = \tfrac{\partial g}{\partial u}$$

Differentiate first, substitute second. $\sin(k\pi) = 0$, $\cos(k\pi) = (-1)^k$. 2×2: $\lambda^2 - \operatorname{tr} A\,\lambda + \det A = 0$; stable ⇔ $\operatorname{tr} A < 0$, $\det A > 0$; saddle ⇔ $\det A < 0$.

Discrete time

$$x[k] = A^k x[0] + \sum_{i=0}^{k-1} A^{k-1-i} B u[i], \qquad A^k = V \Lambda^k V^{-1}, \qquad A^k v = \lambda^k v.$$

Asymptotically stable ⇔ all $|\lambda| < 1$. Impulse response $x[k] = A^{k-1}B$; unbounded ⇒ not BIBO stable.